K-means Clustering

O'nel Hounnouvi

1 Modélisation et objectif

Le problème des k-moyennes (*k-means* en anglais) est un problème d'optimisation combinatoire de partitionnement de données. On dispose d'un ensemble de points $\{x_1, ..., x_n\} \subseteq \mathbb{R}^d$ et d'un entier $k \in \{1, \ldots, n\}$ (nombre de clusters). Le problème consiste à trouver une partition de ces points en k groupes $C_1, \ \ldots, \ C_k$ appelés clusters. Chaque cluster est représenté par son centre μ_j appelé centroïde.

Il s'agit donc de minimiser la somme des carrés intra-cluster (WCSS, *within-cluster sum of squares*), c'est-à-dire la somme des distances au carré entre chaque point et le centroïde de son cluster :

$$\min_{C_1, \ ..., \ C_n} \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

2 Dérivation

Deux sous-problèmes se résolvent en fermé : le choix des centroïdes à partition fixée, et le choix de la partition à centroïdes fixés.

Centroide optimal, à affectation fixée Soit $C_1, \ \ldots, \ C_k$ une partition fixée de $\{x_1, ..., x_n\}$. On a la fonction de coût associée:

$$J(\mu_1, \ldots, \mu_k) = \sum_{j=1}^{k} J_j(\mu_j), \qquad J_j(\mu_j) = \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Chaque J_j ne dépend que de μ_j ; on peut donc minimiser J terme par terme. On a, $\nabla_{\mu_j} \|x_i - \mu_j\|^2 = -2(x_i - \mu_j)$, donc par linéarité :

$$\nabla_{\mu_j} J_j(\mu_j) = \sum_{x_i \in C_j} -2(x_i - \mu_j) = 0$$

$$\sum_{x_i \in C_j} x_i - |C_j| \, \mu_j = 0$$

$$\mu_j^\star = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

La hessienne vaut $\nabla^2 J_j = 2|C_j| \, I_d \succ 0$: J_j est strictement convexe, le point critique est donc bien l'unique minimum. Le centroide optimal est donc la moyenne des points de son cluster.

Affection optimale à centroides fixés A $\mu_1, \ldots, \mu_k$ fixés, notons $c(i)$ le cluster assigné au point x_i. La fonction de perte s'écrit

$$J(c) = \sum_{i=1}^{n} \left\| x_i - \mu_{c(i)} \right\|^2$$

Elle se décompose en n termes indépendants, un par point: le terme pour x_i ne dépend que de $c(i)$ et jamais de l'affectation des autres points. Minimiser $J(c)$ sur toutes les affectations possibles revient donc à minimiser chaque terme séparément:

$$c^\star(i) = \arg \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|^2$$

Cela revient à assigner chaque point au centroide le plus proche, ce dernier étant le représentant du cluster.

Le problème global Trouver le partitionnement optimal est NP-difficile, même pour $k = 2$ en dimension quelconque (Aloise et al., 2009), ou pour $d = 2$ à k quelconque (Mahajan et al., 2009). C'est la raison pour laquelle on utilise une heuristique: l'algorithme de Lloyd (1982).

3 Algorithme de Lloyd

L'algorithme de Lloyd exploite directement les deux résultats précédents en alternant les deux étapes : affecter chaque point à son centroïde le plus proche (bloc c, à μ fixé), puis recalculer chaque centroïde comme la moyenne de son cluster (bloc μ, à c fixé). Il s'agit d'un algorithme itératif de type descente par coordonnées par blocs.

Algorithm 1: Algorithme de Lloyd (K-means)

Input: points $x_1, \ldots, x_n \in \mathbb{R}^d$, nombre de clusters k, itérations max $T_{\max}$, tolérance ε
Output: centroïdes $\mu_1, \ldots, \mu_k$

Initialiser $\mu_1, \ldots, \mu_k$ `// aléatoire ou k-means++ (algorithme 2)`
for $t = 1, \ldots, T_{\max}$ **do**
  for $i = 1, \ldots, n$ **do**
    $c(i) \leftarrow \arg\min_j \|x_i - \mu_j\|^2$ `// étape d'affectation`
  $\mu^{\text{old}} \leftarrow \mu$
  for $j = 1, \ldots, k$ **do**
    $C_j \leftarrow \{x_i : c(i) = j\}$
    if $C_j \neq \emptyset$ **then**
      $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$ `// étape de mise à jour`
  if $\max_{j,l} |\mu_{j,l} - \mu_{j,l}^{old}| \leq \varepsilon$ **then**
    break
return $\mu_1, \ldots, \mu_k$

Remarques. Le critère d'arrêt de cette implémentation porte sur les centroïdes (`np.allclose`, avec une tolérance ε à 10^{-5} par défaut). Il peut aussi porter sur les affectations, si elles ne changent plus d'une itération à l'autre. A convergence exacte les deux sont équivalents (si les affectations ne changent plus, les centroïdes non plus).

4 Initialisation

Le résultat dépend de l'initialisation des centroïdes $\mu_1^{(0)}, \dots, \mu_k^{(0)}$. L'initialisation *aléatoire* tire k points distincts uniformément parmi les x_i. Cela peut conduire à un mauvais partitionnement.

L'initialisation *k-means++* (Arthur & Vassilvitskii, 2007) tire le premier centroïde uniformément, puis chaque centroïde suivant parmi les points, avec une probabilité proportionnelle à leur distance au carré au centroïde déjà choisi le plus proche (algorithme 2) :

$$\mathbb{P}(\mu_j = x_i) = \frac{D(x_i)^2}{\sum_{m=1}^{n} D(x_m)^2}, \qquad D(x)^2 = \min_{l<j} \|x - \mu_l\|^2$$

Les points éloignés des centroïdes déjà choisis ont donc plus de chances d'être tirés, ce qui disperse les graines.

Algorithm 2: Initialisation k-means++

Input: points $x_1, \dots, x_n \in \mathbb{R}^d$, nombre de clusters k
Output: centroïdes initiaux $\mu_1^{(0)}, \dots, \mu_k^{(0)}$

Tirer $\mu_1^{(0)}$ uniformément parmi $x_1, \dots, x_n$
for $j = 2, \dots, k$ **do**
  for $i = 1, \dots, n$ **do**
    $D(x_i)^2 \leftarrow \min_{l<j} \|x_i - \mu_l^{(0)}\|^2$
  Tirer $\mu_j^{(0)}$ parmi les x_i avec la probabilité $D(x_i)^2 / \sum_m D(x_m)^2$
return $\mu_1^{(0)}, \dots, \mu_k^{(0)}$

Remarque : Cette initialisation garantit, avant même d'appliquer Lloyd (qui ne peut ensuite que faire décroître J), $\mathbb{E}[J] \leq 8(\ln k + 2)\, J_{\text{opt}}$ où J_{opt} est le coût optimal.

5 Convergence

Notons $J^{(t)} = J\big(c^{(t)}, \mu^{(t)}\big)$ le coût à l'issue de l'itération t. Chacune des deux étapes est une minimisation *exacte* de son sous-problème, pas une approximation :

$$J\big(c^{(t)}, \mu^{(t-1)}\big) \leq J\big(c^{(t-1)}, \mu^{(t-1)}\big) \text{ (affectation)}, \qquad J\big(c^{(t)}, \mu^{(t)}\big) \leq J\big(c^{(t)}, \mu^{(t-1)}\big) \text{ (mise à jour)}$$

donc $J^{(t)} \leq J^{(t-1)}$: le coût ne peut que décroître ou stagner.

Terminaison en temps fini. Si l'affectation change à l'itération t, c'est qu'au moins un point est devenu strictement plus proche d'un autre centroïde : la première inégalité est alors stricte et $J^{(t)} < J^{(t-1)}$. Après la mise à jour, $J^{(t)}$ ne dépend plus que de la partition $C^{(t)}$ (les centroïdes en sont les moyennes) : une partition ne peut donc jamais réapparaître. Comme il n'y a qu'un nombre fini de partitions des n points en k groupes, l'algorithme atteint en un nombre fini d'itérations une partition qui ne change plus, c'est-à-dire des centroïdes fixes. Il s'arrête donc.

Pas d'optimalité globale. En ce point fixe, aucune des deux étapes ne peut plus diminuer J, mais ce n'est pas forcément l'optimum global : le problème joint (sur c discret et μ continu) n'est pas convexe, seulement convexe par blocs. Le résultat dépend donc fortement de l'initialisation $\mu^{(0)}$, d'où l'intérêt de k-means++ et des relances multiples.

6 Complexité

À chaque itération, l'étape d'affectation calcule nk distances, chacune en $O(d)$, soit $O(nkd)$; l'étape de mise à jour coûte $O(nd)$ (chaque point entre une fois dans la moyenne de son cluster).

Une itération coûte donc $O(nkd)$, et l'algorithme $O(T\,nkd)$ au total, avec $T \leq T_{\max}$ le nombre d'itérations effectuées.

L'initialisation k-means++ de l'implémentation actuelle recalcule, pour chacun des $k-1$ centroïdes à tirer, la distance de chaque point à tous les centroïdes déjà choisis : $\sum_{j=1}^{k-1} n\,j\,d = O(nk^2d)$. On peut la ramener à $O(nkd)$ en ne mettant à jour la distance minimale $D(x_i)^2$ qu'avec le dernier centroïde ajouté, au lieu de la recalculer sur tous.

Références

[1] Aloise, Deshpande, Hansen, Popat; *NP-hardness of Euclidean sum-of-squares clustering*, Machine Learning, 2009.

[2] Mahajan, Nimbhorkar, Varadarajan; *The planar k-means problem is NP-hard*, WALCOM, 2009.

[3] Lloyd; *Least squares quantization in PCM*, IEEE Transactions on Information Theory, 1982.

[4] Arthur, Vassilvitskii; *k-means++: the advantages of careful seeding*, SODA, 2007.

[5] Slides d'un cours de Data Science, Oxford; `https://www.stats.ox.ac.uk/~cucuring/CDT_13_Clustering_kMeans_SpecClust_Cheeger_FoDS.pdf`